

MNIST 单隐藏层神经网络宽度 准确率与 CPU 推理延迟实证研究

可复现实验技术报告

2026 年 9 月

摘要

本报告在 MNIST 上考察单隐藏层多层感知机的隐藏层宽度如何影响测试准确率、模型资源量与单样本 CPU 推理延迟。实验覆盖 $h \in \{4, 8, 16, 32, 64, 128, 256\}$ ，每个宽度使用 3 个随机种子训练，并以 $h=256$ 为参考。结果显示，准确率随宽度增加而提升，但在 $h \geq 64$ 后边际收益明显减弱： $h=64$ 的平均准确率为 97.38%，仅比参考模型低 0.50 个百分点，同时减少 75.00% 的 MACs，CPU 中位延迟降低 12.06%； $h=32$ 的平均准确率为 96.95%，下降 0.93 个百分点，但减少 87.50% 的 MACs。实测延迟收益显著小于理论计算量收益，说明小型模型上的固定框架开销和内存访问不可忽略。该实验可作为后续动态宽度、资源受限推理与置信度风险研究的可复现基线。

关键词：MNIST；多层感知机；隐藏层宽度；准确率—效率权衡；推理延迟；可复现实验

核心发现

- 稳健折中： $h=64$ 以 25% 的参数与 MACs 保留参考模型 99.49% 的绝对准确率水平。
- 激进折中： $h=32$ 在准确率下降 0.93 个百分点时，将 MACs 降至参考模型的 12.50%。
- 计算量不等于实际延迟： $h=64$ 虽减少 75% 的 MACs，中位延迟仅降低 12.06%。
- 极窄模型不稳定： $h=4$ 的准确率标准差为 4.06 个百分点，明显高于其他宽度。

1 研究问题与定位

本研究回答一个具体的工程问题：当一个全连接分类器被缩窄时，能够以多大的准确率代价换取多少理论计算量和真实 CPU 延迟收益？研究重点不是提出新算法，而是建立一条完整、可复查的实验链路，展示宽度缩减带来的收益、收益递减现象，以及理论资源指标与端侧实测延迟之间的差异。

候选模型只在隐藏层宽度上变化，其余训练与评估设置保持一致。报告采用 $h=256$ 作为已测试范围内的参考模型，并分别考察平均测试准确率下降不超过 0.5 和 1.0 个百分点时的最小候选宽度。阈值结果是当前候选集中的经验结论，不应解释为对未测试宽度的最优性证明。

2 实验设计

2.1 数据与模型

MNIST 训练集被固定划分为 54,000 个训练样本与 6,000 个验证样本，测试集包含 10,000 个样本。输入仅执行 ToTensor 与均值 0.1307、标准差 0.3081 的标准化，不使用数据增强。模型结构为 784 维展平输入、一个 ReLU 隐藏层与 10 类线性输出层。

对隐藏层宽度 h ，参数量为 $P(h)=784h+h+10h+10=795h+10$ ；忽略偏置加法时，每个样本的近似乘加次数为 $MACs(h)=784h+10h=794h$ 。因此，除常数输出偏置外，参数量与理论计算量都随 h 近似线性增长。

2.2 训练与评估协议

项目	设置
宽度	4、8、16、32、64、128、256
随机种子	42、43、44
优化器	Adam，学习率 0.001
批量大小	128
模型选择	验证损失最低的检查点
提前停止	耐心值 5；一般上限 30 轮
$h=4$ 诊断	上限延长至 60 轮，避免把未收敛误判为容量不足
训练设备	Apple MPS；不可用时自动回退 CPU

2.3 延迟测量

延迟测试在 CPU、单线程、batch size=1 条件下进行。每个模型先预热 1,000 次，再执行 10,000 次计时，报告中位延迟、P95 延迟和平均延迟。该测量不包含数据读取与预处理，反映的是 PyTorch eager 模式下模型前向传播的微基准性能。

2.4 运行环境

类别	说明
硬件	MacBook Neo, Apple A18 Pro, 6 核 CPU, 8 GB 内存
系统	macOS 26.6.2, arm64
软件	Python 3.11.16, PyTorch 2.14.0, MPS 可用
结果文件	原始结果、汇总表、延迟表与绘图脚本均随仓库提供

3 宽度与测试准确率

平均测试准确率从 h=4 的 84.72% 提升到 h=256 的 97.88%。提升主要集中在较窄区间：h=4 到 h=32 累计提升 12.23 个百分点，而 h=64 到 h=256 仅再提升 0.50 个百分点，呈现明显的边际收益递减。

h	测试准确率 (%)	最佳轮次均值	参数量	MACs/样本
4	84.72 ± 4.06	30.33	3,190	3,176
8	92.62 ± 0.06	21.00	6,370	6,352
16	95.15 ± 0.29	20.67	12,730	12,704
32	96.95 ± 0.08	14.67	25,450	25,408
64	97.38 ± 0.09	10.33	50,890	50,816
128	97.79 ± 0.12	9.33	101,770	101,632
256	97.88 ± 0.19	7.67	203,530	203,264

表 1 不同隐藏层宽度的准确率与资源规模 准确率为 3 个随机种子的均值±1 个标准差

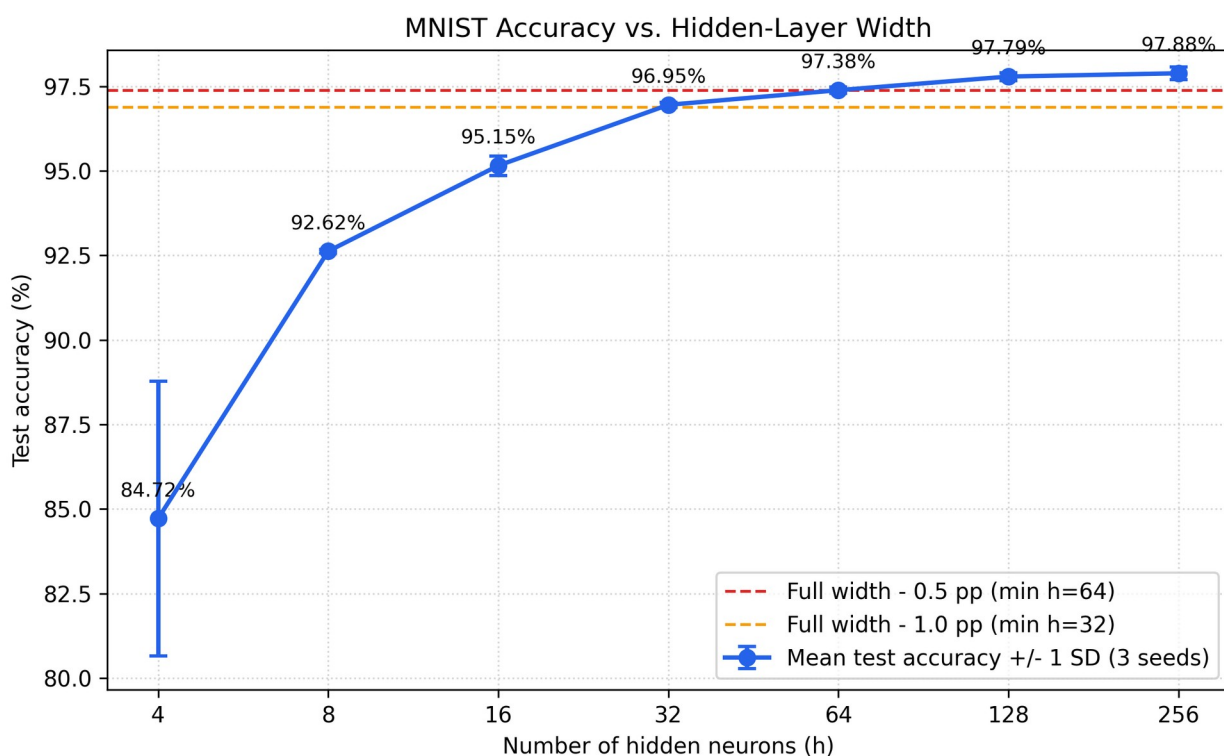


图1 隐藏层宽度与测试准确率 虚线分别表示相对 h=256 下降 0.5 和 1.0 个百分点的阈值

3.1 阈值选择

以 h=256 的平均准确率 97.88% 为参考，允许下降 0.5 个百分点时，目标准确率为 97.38%，候选集合中的最小宽度为 h=64；允许下降 1.0 个百分点时，目标准确率为 96.88%，最小宽度为 h=32。h=128 相对参考仅下降 0.10 个百分点，但只将资源量减半，其折中力度弱于 h=64。

3.2 随机种子稳定性

h≥8 时，各宽度的标准差不超过 0.29 个百分点，结果较稳定。h=4 的标准差达到 4.06 个百分点，其中 seed 44 的测试准确率为 80.03%，而另外两个种子约为 87.1%。这表明极窄网络对初始化和优化路径敏感；用单次运行评价极窄模型会产生误导，因此报告保留 3 个种子的完整结果，并对 h=4 使用更长的 60 轮上限。

4 准确率—资源—延迟权衡

理论 MACs 随宽度等比例下降，但实测 CPU 延迟下降得更慢。h=4 的 MACs 比 h=256 少 98.44%，中位延迟却只降低 15.96%；h=64 减少 75.00% 的 MACs，中位延迟降低 12.06%。当矩阵乘法已经很小时，Python 与框架调度、算子启动、张量元数据处理和内存访问等固定成本占比上升，因此 MACs 不能直接替代目标设备上的延迟测量。

h	中位延迟 (μ s)	P95 (μ s)	延迟降低	准确率下降 (pp)
4	9.875	10.167	15.96%	13.16
8	9.916	10.083	15.61%	5.26
16	9.916	10.125	15.61%	2.73
32	10.166	10.417	13.48%	0.93
64	10.333	10.542	12.06%	0.50
128	10.917	11.209	7.09%	0.10
256	11.750	11.958	0.00%	0.00

表 2 单样本 CPU 延迟与相对 h=256 的变化

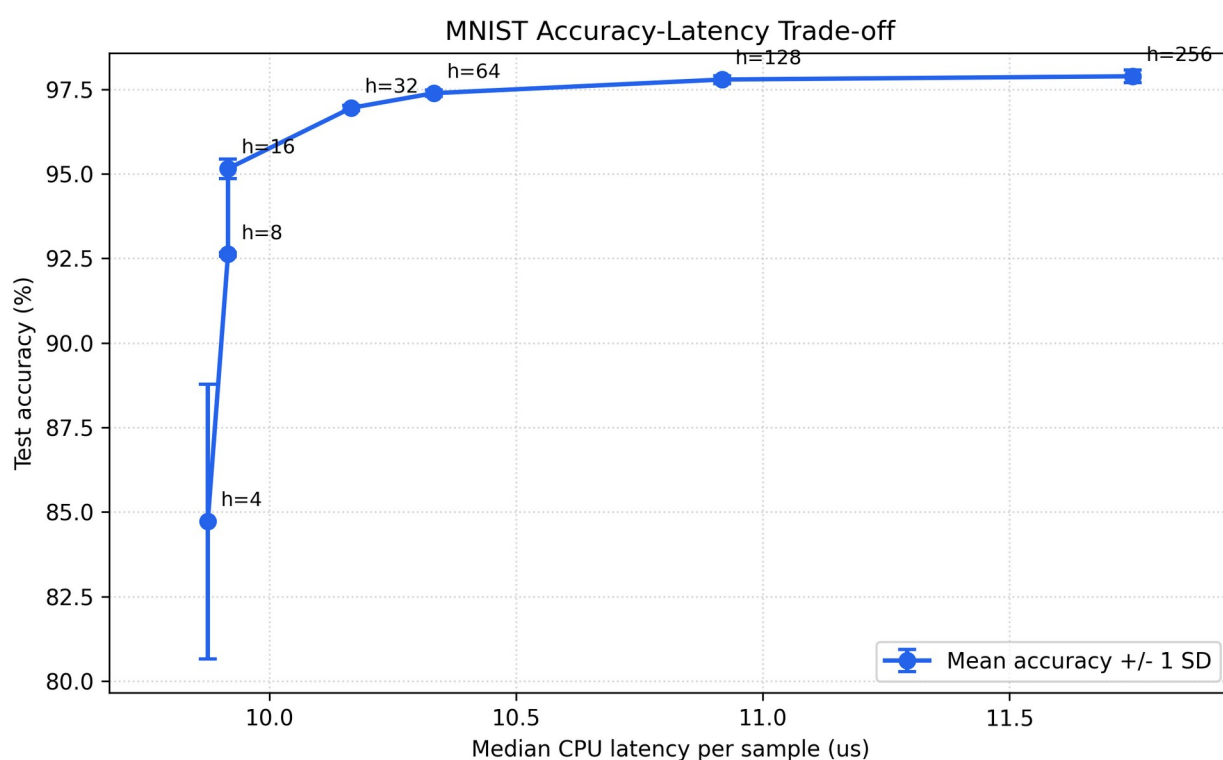


图 2 平均测试准确率与单样本 CPU 中位延迟的经验权衡曲线

4.1 推荐工作点

- 默认建议 h=64：平均准确率 97.38%，标准差 0.09 个百分点；MACs 减少 75.00%，中位 CPU 延迟降低 12.06%。
- 资源更紧张时选 h=32：平均准确率 96.95%；MACs 减少 87.50%，中位延迟降低 13.48%。
- 更重视精度时选 h=128：平均准确率 97.79%，仅下降 0.10 个百分点，但延迟改善只有 7.09%。

5 讨论

5.1 结果如何解释

本实验说明，模型宽度与准确率之间并非线性关系。较窄区间增加少量隐藏单元即可显著提升表示能力，而达到 $h=64$ 后继续翻倍宽度主要换来不到 0.5 个百分点的提升。对这一任务与模型而言， $h=64$ 位于经验“拐点”附近：它在跨种子稳定性、准确率和资源规模之间取得了较均衡的结果。

另一方面，理论压缩比不能直接视为端侧加速比。 $h=64$ 的理论乘加量只有参考模型的 25%，但单样本 CPU 延迟仍为参考模型的 87.94%。如果目标是部署，后续实验应优先在真实推理后端与目标批量大小上测量，并同时报告中位数与尾延迟。

5.2 可用于后续研究的方向

- 动态宽度：按样本置信度在 $h=32$ 、64、128 之间切换，并把平均计算预算作为约束。
- 风险校准：比较最大软最大概率、熵或温度缩放能否可靠触发更宽模型。
- 更真实的推理后端：比较 PyTorch eager、TorchScript、Core ML 或其他编译后端的固定开销。
- 更复杂任务：在 Fashion-MNIST、CIFAR 或卷积网络上验证拐点是否仍存在。

6 局限与有效性威胁

- 任务与结构较简单。MNIST 上的单隐藏层 MLP 不能代表现代视觉模型，其数值结论不应直接外推到卷积网络或 Transformer。
- 随机性覆盖有限。每个宽度只有 3 个种子，足以暴露 $h=4$ 的不稳定性，但不足以给出高精度置信区间。
- 延迟只来自一台机器。结果依赖 Apple A18 Pro、CPU 单线程、PyTorch eager 与系统状态；它不是通用硬件排名。
- 微基准范围有限。计时不含数据加载、归一化、进程启动和模型加载，因此不能等同于端到端应用时延。
- 阈值分析使用测试集均值做描述性比较。若把宽度选择纳入正式模型开发流程，应在验证集上确定阈值，只对最终方案进行一次测试集评估。
- $h=4$ 采用 60 轮上限，其余宽度为 30 轮。该调整用于诊断极窄网络的收敛问题，已在配置和原始结果中明确披露。

7 结论

在本实验条件下，隐藏层宽度从 4 增加到 256 会持续提高 MNIST 测试准确率，但 $h=64$ 之后的边际收益很小。若希望将相对 $h=256$ 的平均准确率损失控制在 0.5 个百分点内， $h=64$ 是已测试候选中的最小宽度，并可减少 75% 的参数与 MACs；若容许约 1 个百分点损失， $h=32$ 可将资源量降低 87.5%。真实 CPU 延迟的改善远小于理论计算量下降，说明高质量的轻量化结论必须同时包含准确率、资源量和目标硬件延迟。

因此，这项工作可以视为一个已经闭环的可复现实证研究：问题明确、训练和评估协议固定、原始数据完整、跨种子结果可汇总、资源与延迟指标可复查。它适合作为课程项目、技术博客或后续动态宽度研究的基线；若要投稿为研究论文，则仍需更强任务、更广硬件、更严格统计检验以及方法层面的新贡献。

8 复现实验

仓库提供环境检查、单次训练、批量宽度实验、准确率分析、CPU 延迟基准和综合权衡分析。

完整复现流程如下：

```
conda env create -f environment.yml
conda activate mnist-study
bash scripts/reproduce.sh
```

仓库已包含本报告使用的原始 CSV、汇总 CSV 与图像，读者即使不重新训练，也可直接运行 `analyze.py` 与 `analyze_tradeoff.py` 复核统计结果和绘图。

参考文献

- [1] LeCun Y, Bottou L, Bengio Y, Haffner P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998, 86(11): 2278–2324.
- [2] Paszke A, Gross S, Massa F, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 2019, 32.